

tju-LPU-tutorial

Tutorial :

Google TPUv1 Architecture

- MAC：乘累加单元
- MXU：TPU的核心，包含 256×256 个MAC单元
- UB：unified buffer，存储输入数据/中间结果，通用片上缓存，以减少MXU的片外访存，降低延迟和功耗
- Weight FIFO：PCIe接口 → TPU DDR（推理前） → Weight FIFO → MXU（运行时）
- Accumulators：→ + → Buffer → feedback → + → Buffer，存储MXU每个时钟产生的Partial Sum
- Activation&Normalize/Pool：加速激活函数/归一化/池化

OpenTPU Architecture

OpenTPU简介

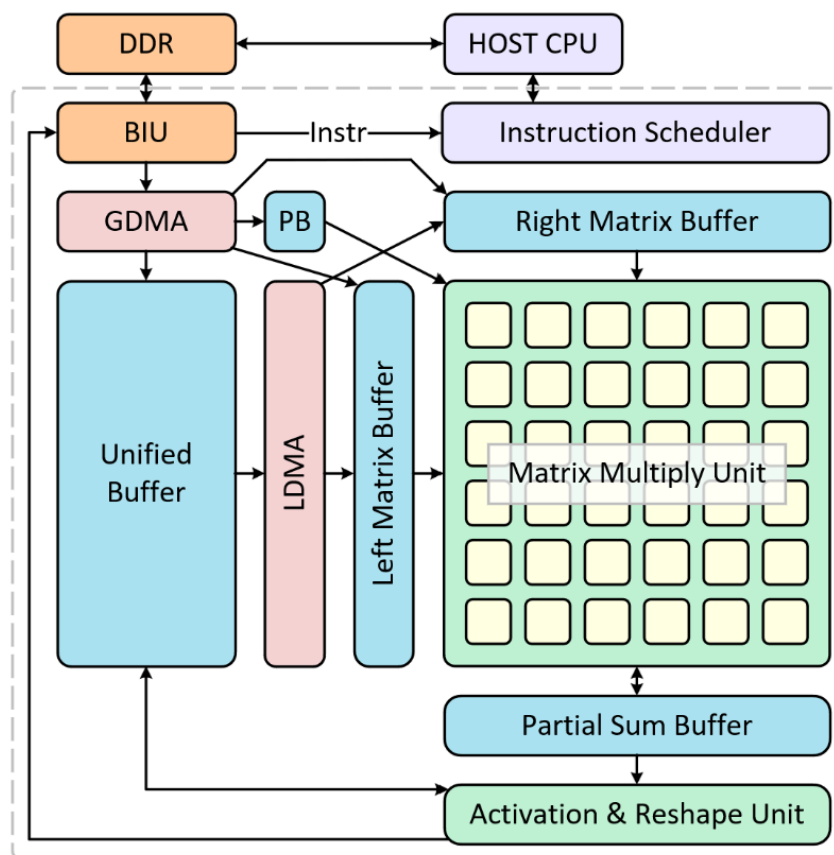


图2 OpenTPU架构

- **BIU** : Bus interface unit : OpenTPU ↔ DDR, 数据和控制Bus接口
- **instr** : instruction → 给submodule的指令, NPU的ISA
- **instruction scheduler** : 中央调度器 (DDR → BIU → Instruction scheduler → submodule)
- **ARU** : activation and reshape unit : 相当于Activation & normalize/pool, 但是可以直通BIU
- **GDMA** : global搬运数据从DDR到片上存储 : DDR → Input Feature Map → UB & DDR → Input weight → Weight Buffer
- **LDMA** : local片上搬运数据, 并改数据layout : 片内实现im2col
- **L/RMB** : left/Right Matrix Buffer, 其中LMB存储im2col后的feature map ; RMB用于存储权重/卷积核参数

- PB : bias和反量化系数
- PSB : partial sum buffer $\leftrightarrow$ accumulator

Mem Model

三级缓存：解决速度差太多的问题，防止高速模块一直空转

- **DDR (global data) $\rightarrow$ BIU $\rightarrow$ GDMA $\rightarrow$ UB (tile) $\rightarrow$ (LDMA $\rightarrow$ reshape) $\rightarrow$ PB/LMB/RMB $\rightarrow$ MXU $\rightarrow$ ARU $\rightarrow$ UB $\rightarrow$ BIU $\rightarrow$ DDR**

Data Layout

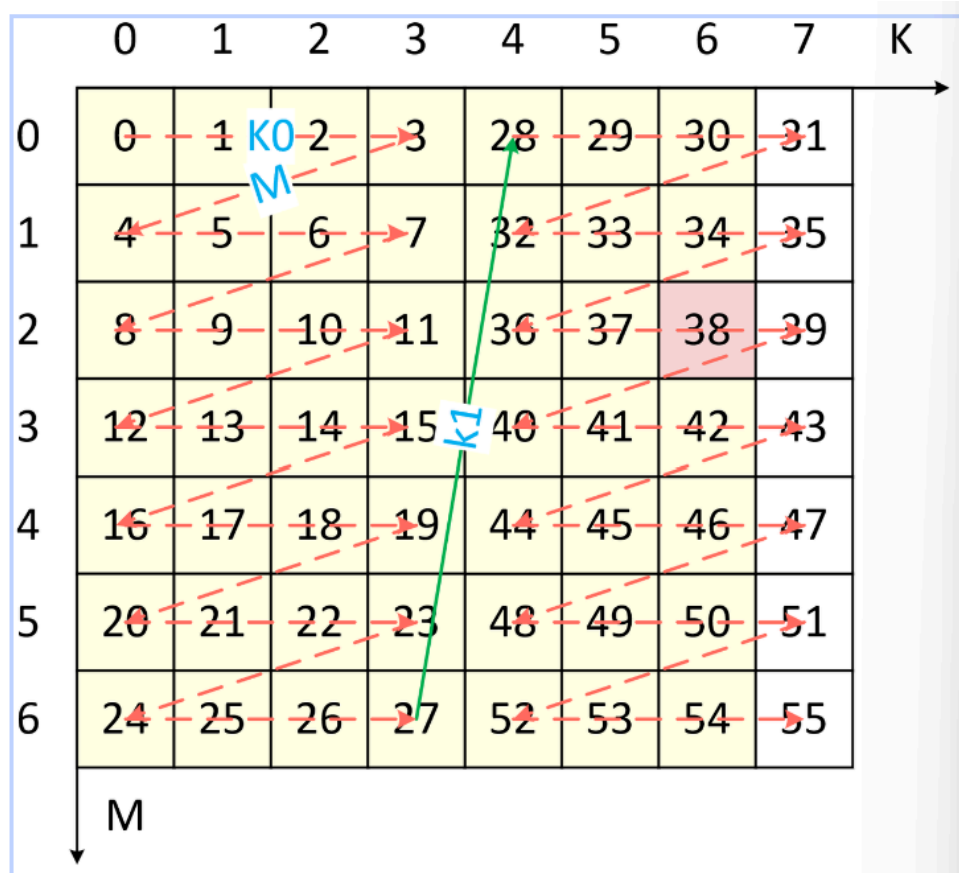
- M $\rightarrow$ Height Direction
- K $\rightarrow$ Width Direction
- MK $\rightarrow$ row scan $\rightarrow$ K dir
- KM $\rightarrow$ col scan $\rightarrow$ M dir
- Summary: 相同的坐标 (m_idx, k_idx) 在不同的Layout中对应着不同的address ;
对于MK : **address=m_idx*K+k_idx*1 (那么stride_m=K, stride_k=1)**

拆轴合轴

- 维度变换
- 拆轴 : e.g. K $\rightarrow$ K1 * K0; K1为循环次数
 - K = 7 K0 = 4 K1 = 2; K维度上补0对齐到K0整数倍
 - 从MK $\rightarrow$ MK1K0 : 先沿着K0方向，再到K1方向，再到M方向，此时**addr = m_idx*K1*K0+k1_idx*K0+k0_idx*1**
- 合轴 $\leftrightarrow$ 拆轴 : 为互逆过程

移轴

- 改变维度上的增长顺序，也改变了对应的stride步长
- 如图所示 : 从 (2, 1, 2) $\rightarrow$ (1, 2, 2) ; addr =
k1_idx*M*K0+m_idx*K0+k0_idx = 1*7*4+2*4+2=38



输入：

$$[B, H_i, W_i, C_i]$$

卷积核：

$$[K_h, K_w, C_i, C_o]$$

输出：

$$[B, H_o, W_o, C_o]$$

其中：

$$H_o = \left\lfloor \frac{H_i + 2P_h - K_h}{S_h} \right\rfloor + 1$$
$$W_o = \left\lfloor \frac{W_i + 2P_w - K_w}{S_w} \right\rfloor + 1$$

卷积公式：

$$out(b, h_o, w_o, co) = \sum_{kh, kw, ci} x(b, h_o + kh, w_o + kw, ci) \cdot w(co, kh, kw, ci)$$
$$out(b, h_o, w_o, co) = \sum_{kh, kw, ci} x(b, h_o + kh, w_o + kw, ci) \cdot w(co, kh, kw, ci)$$

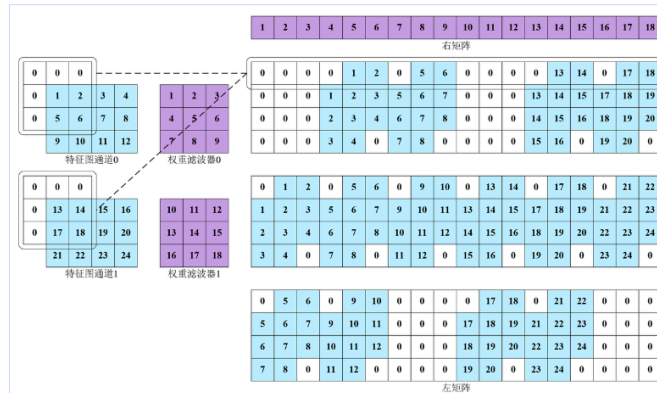
- 所以可以看出，本质上特征图与权重的卷积变成矩阵乘法就是：
 - 对于特征图，每一个输出位置提取一个卷积窗口patch，**[B,Hi,Wi,Ci]** → **(BHoWo)*(KhKwCi)**第一项表示有多少个Patch，第二项表示每个patch的展开长度；
 - 对于权重**[Co,Kh,Kw,Ci]** → **(KhKwCi)*Co**第一项表示卷积核展开长度，第二项表示卷积核个数；矩阵乘法即可得到[B,Ho,Wo,Co]；
 - 这里的BHoWo表示多个维度**flatten成一个维度**

OpenTPU Data Layout (Conv → Matrix Multiplication)

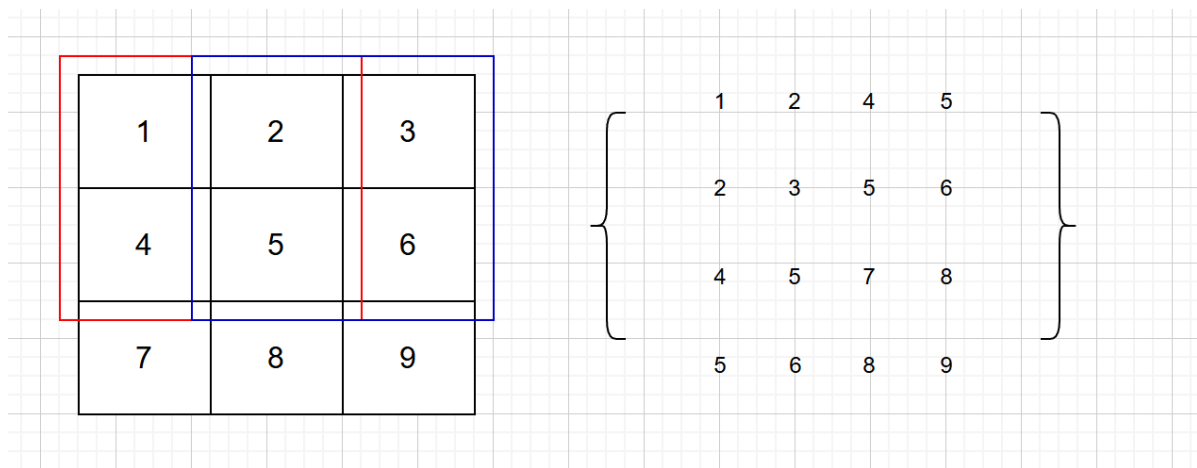
OpenTPU的Data Layout

	Convolution Feature Map	Convolution Weight	Convolution Parameter	Matrix Multiplication Left Matrix	Matrix Multiplication Right Matrix	Matrix Multiplication Parameter
DDR	BHWC/C1BHWCO	CoCiKwKh	Co	MK/K1MK0	NK/K1NK0	N
L1	C1BHWCO			K1BMK0		
L0-IN	LMB: M1K1M0K0 $M1 = \text{ceil_div}(BHW, M0)$ K1=C1KhKw M0=M0 K0=C0	RMB: N1K1K0N0 N1=Co1 K1=Ci1KhKw N0 = Co0 K0=Ci0	PB: N1N0 N1=Co1 N0 = Co0	LMB: M1K1M0K0	RMB: N1K1K0N0	PMB: N1
L0-Out	PSB: M1N1M0N0			PSB: M1N1M0N0		

- 两个视角：在矩阵乘法视角，所有数据格式都被抽象成矩阵，所以DDR中存入MK/NK；在真实世界视角，DDR中存入的实际上是feature map的BHWC，kernel的CoCiKwKh
- 图中卷积layout和矩阵layout表示的值是不变的，这是两种不同的表示方式，两者一一对应
- DDR：可以是BHWC，也可以直接是C1BHWCO（MXU要求的C是一定的，值为C0 $\Leftrightarrow$ K0），这样可以节省一步硬件reshape
- L1（UB）
- L0-IN（MXU）：这里 $M1 = \text{ceil_div}(BHW, M0)$ 是否有误？我觉得应该为 BH_outW_out ，因为conv $\rightarrow$ matrix 应该是变换成 $M \times K$ ，其中K的含义是每个patch，而M的含义是patch的个数，应该为输出Feature Map的像素个数；
 - conv $\rightarrow$ matrix 本质上就是将 feature map reorganize to matrix (im2col $\rightarrow$ image to col：将 feature map提取出window[1,2;4,5] $\rightarrow$ [1,2,4,5] $\rightarrow$ 从而转换成matrix：[1,2,4,5];[2,3,5,6].....)



- LMB按行给MXU；而RMB按列给MXU；适配传播方向，所以小tile中LMB是M0K0，RMB是K0N0；**RMB LMB的大tile数据格式为什么是M1K1，K1N1？**



- L0-OUT (MXU)

Exec Model

多模块并行执行：同步数据机制为set和wait标志位，各个执行模块被instruction scheduler控制

- set flag → 通知下级模块数据搬运完成，他是一个completion event标志位，并不会阻塞scheduler硬件指令调度器
- wait flag → 阻塞scheduler，等待flag值被置1

operator & task

算子是神经网络里的计算节点：e.g.：conv2d relu softmax...

算子是算法层面的语义，但是没有具体到硬件实现：e.g.：

Conv2d(Cin=1024,Cout=512,Kernel=3,Stride=2)

一个operator需要编译/映射的过程 → conv需要tile成很多task (start → instructions → end) → Hardware Module

taskinfo

taskinfo用于给instruction scheduler读取，是调度**入口信息**，三个字段分别为start指令地址、指令数量、ID

workflow

1. openTPU读取的是打包的**taskinfo**包，根据taskinfo包的信息 读取DDR中编译好的task中的**instr**
2. openTPU 在 当前task计算完成/执行完全部指令后 向cpu发送**中断**

programflow

需求分析

架构定义：处理器、Mem Model、Exec Model

ISA定义：instruction集合

ISA的Func Model验证：cpp/py → 验证定义的ISA能不能满足预期的需求

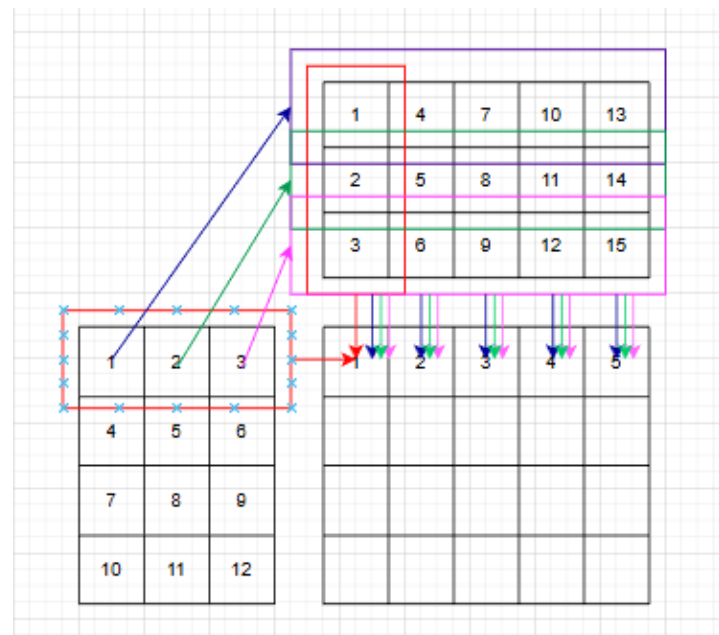
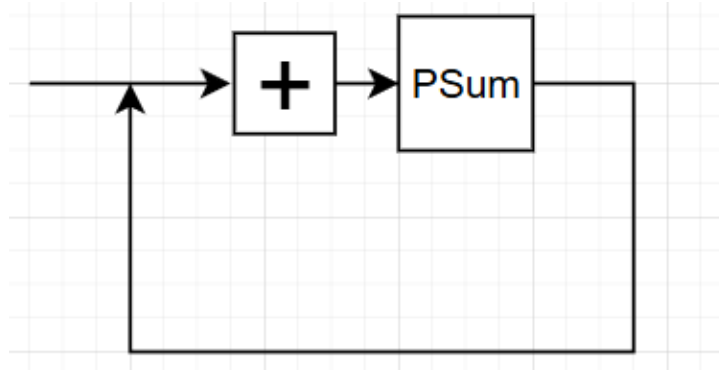
微架构设计：微架构是为了实现ISA而具体搭建的硬件结构：例如GDMA LDMA MXU ARU Scheduler以及接口互联

微架构的CAModel验证：cpp描述微架构，是Cycle-Accurate型仿真，需要模拟硬件时钟周期，一笔一笔操作数据，验证微架构是否满足预期需求，FModel是CAModel的Golden；CAModel是RTL的Golden

RTL实现：使用验证的微架构进行RTL设计实现

矩阵分块

大矩阵的乘法可以切分成小矩阵的乘法，矩阵乘法分为内积和外积



他们结果一致： $C1 = A1B1 + A2B2 + A3B3$ ，但是计算顺序不同，其中内积是 $A1B1 \rightarrow A2B2 \rightarrow A3B3$ ，需要的PSum少 = 1，但是单元无法复用，相当于是**时间换空间**；外积是 $A1B1 \rightarrow A1B4 \rightarrow A1B7 \rightarrow A1B10 \rightarrow A1B13 \rightarrow \dots$ ，需要的PSum多 = 5，但是单元可以复用，相当于是**空间换时间**。

矩阵乘法与多级缓存

L2 (DDR) $\rightarrow$ L1 (UB) $\rightarrow$ L0 (LMB/RMB) $\rightarrow$ MXU

L2 $\rightarrow$ 存储完整的矩阵

L1 $\rightarrow$ 存储优化；存储稍大的tile，尽量减少DDR访问，要尽可能利用存储空间，L1不要求必须是M0/M_L0的整数倍，因为这样会造成存储空间浪费（e.g：UB_max = 300, m0 = 64, UB_valid = 256，剩余44，这些44就是浪费）

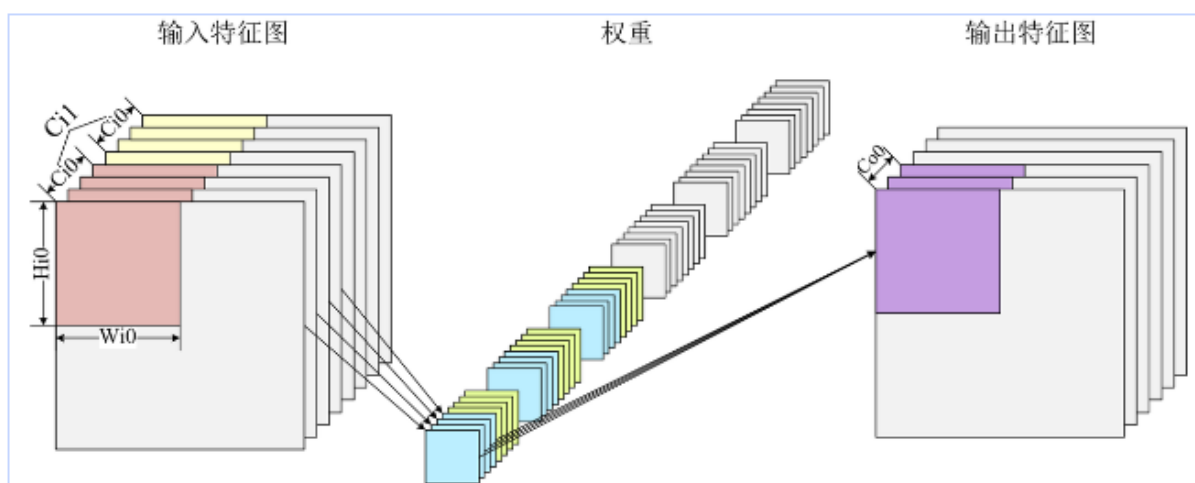
LO → **计算优化**；是更小的tile，必须是MXU要求输入的整数倍（尾块Padding补0），避免MXU来补0，减少硬件复杂度

MXU → 固定了M0, N0, K0

卷积

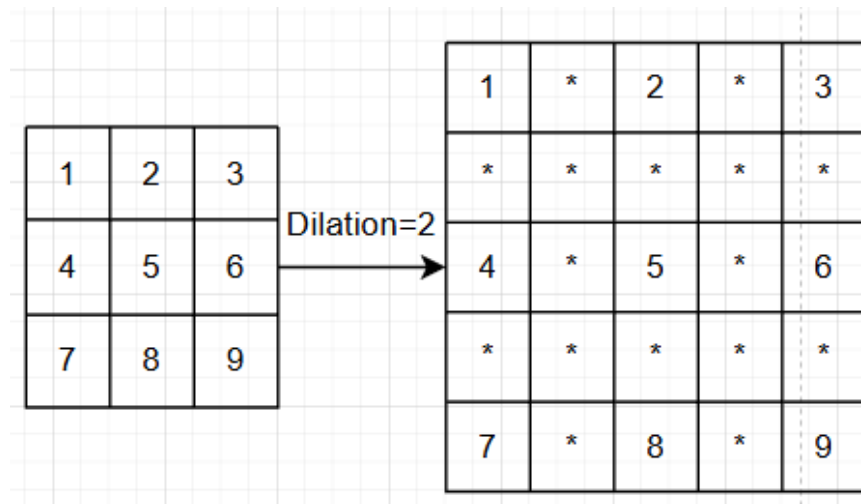
讨论真实存储的layout**如何进行tile**，reorganize成MXU可以接受的matrix layout

tile策略：

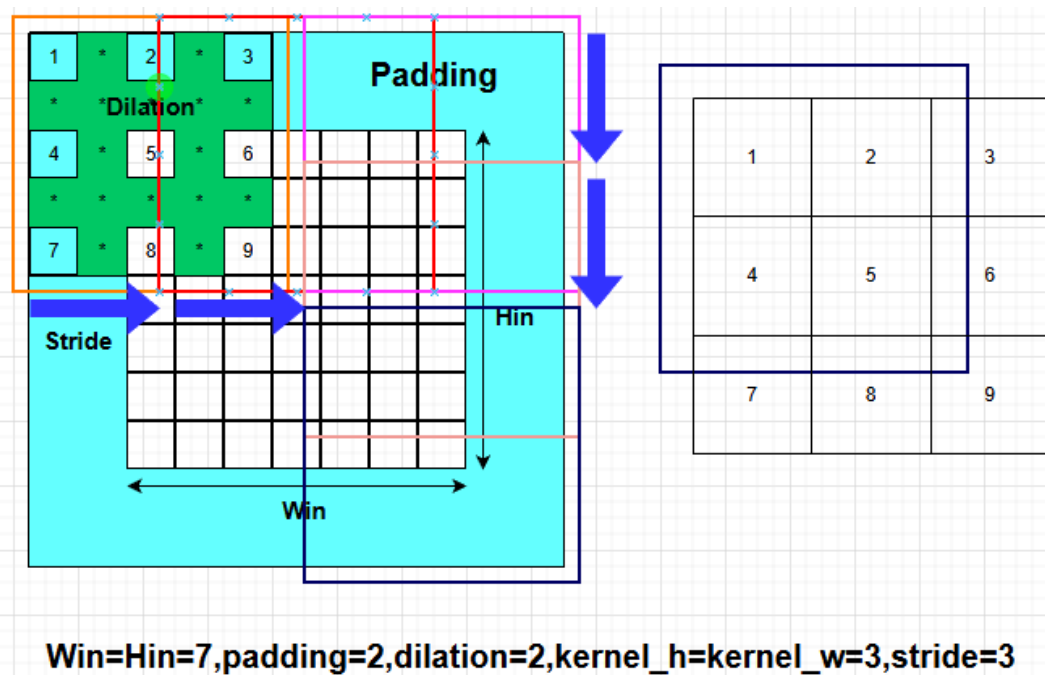


- K_h 和 K_w 较小，不切分
- 输入特征图的 H_i , W_i , C_i 维度切分成： H_{i0} , W_{i0} , C_{i0}
- 输出特征图 C_o 维度/Kernel个数切分： C_{o0} → 由于 C_o 和 C_i 都进行了切分，所以只有前 C_{o0} 个Kernel的前 C_{i0} 个维度 C_a 参与了convolution计算
- → 现在可以重新解释 ****1: H_{i1} , W_{i1} , C_{o1} , C_{i1}** ，例如 $C_{i1} = \text{ceil_div}(C_i, C_{i0})$ 表征切分成了多少个 C_{i0} ，如果 C_i 不是 C_{i0} 整数倍，需要将 C_i 进行补0到 C_{i0} 的整数倍
- tile实现：
 - C_o ：卷积在这个方向上，各个输出Channel之间相互独立，没有依赖关系，可以分次计算。
 - C_i ：在 C_i 方向切分后，需要进行累加，算完最后一个tile才能得到完整输出
 - HW：考虑dilation、stride和pad三个参量

- dilation：空洞率 → 表示**采样间距**，空洞卷积中，采用dilation在不增加参数量，不降低分辨率的情况下扩大感受野



- stride：步长
- pad：填充
- 计算input feature map的起始和结束坐标：



- 输入特征图、权重、参数、输出特征图维度关系

- 假设有 $K_h = Kw$; 则 $K_{h/w\text{eff}} = \text{Dilation} \times (K_{h/w} - 1) + 1$; 这是因为有 $K - 1$ 个gap, 而每个gap的长度是 D , 再加1 (第一个点) 就可以表示卷积核的有效宽/高。

- 假设 $K = 3, D = 1 \rightarrow x \times x$ 相邻点距离1

- $H_o = \frac{H_i + \text{padding} \times 2 - K_{heff}}{\text{stride}} + 1$; 其中分子表示最大合法起点, 即 $\text{Input} - K_{eff}$; 每次跳 stride 步; 分式表示还能往后移动多少次, 最终要加1 (第一个位置)

- 同理 $W_o = \frac{W_i + \text{padding} \times 2 - K_{weff}}{\text{stride}} + 1$

- **输入特征图、输出特征图坐标关系**

- 原始点为0, $h_i, h_o, w_i, w_o, k_h, k_w$ 表示坐标

- 假设输出 $o(2, 3) \rightarrow \text{start} : i(2, 3), \text{Kernel}_h/w = 3 \rightarrow \text{end} : i(2 + 3, 3 + 3) = i(5, 6)$

- 即 $h_i = h_o + k_h; w_i = w_o + k_w$

- 加 stride , 输出每移动1格, 输入移动2格, **窗口起点偏移** 即 $o(2, 3) \rightarrow \text{start} : i(2 \times (2 - 0) + 0, 2 \times (3 - 0) + 0) = i(4, 6) \rightarrow \text{end} : i(4 + 3, 6 + 3)$

- 即 $h_i = h_o \times \text{stride} + k_h; w_i = w_o \times \text{stride} + k_w$

- 加 padding , 输入变为 $-1 \ 0 \ 1 \ 2 \dots$, 也就是说需要**减去填0后的坐标偏移**

- 即 $h_i = h_o \times \text{stride} - \text{padding} + k_h; w_i = w_o \times \text{stride} - \text{padding} + k_w$

- 加 dilation , **Kernel内部偏移**, 其中的最大坐标点为 $\text{kernel}_h \times \text{dilation}; \text{kernel}_w \times \text{dilation}$

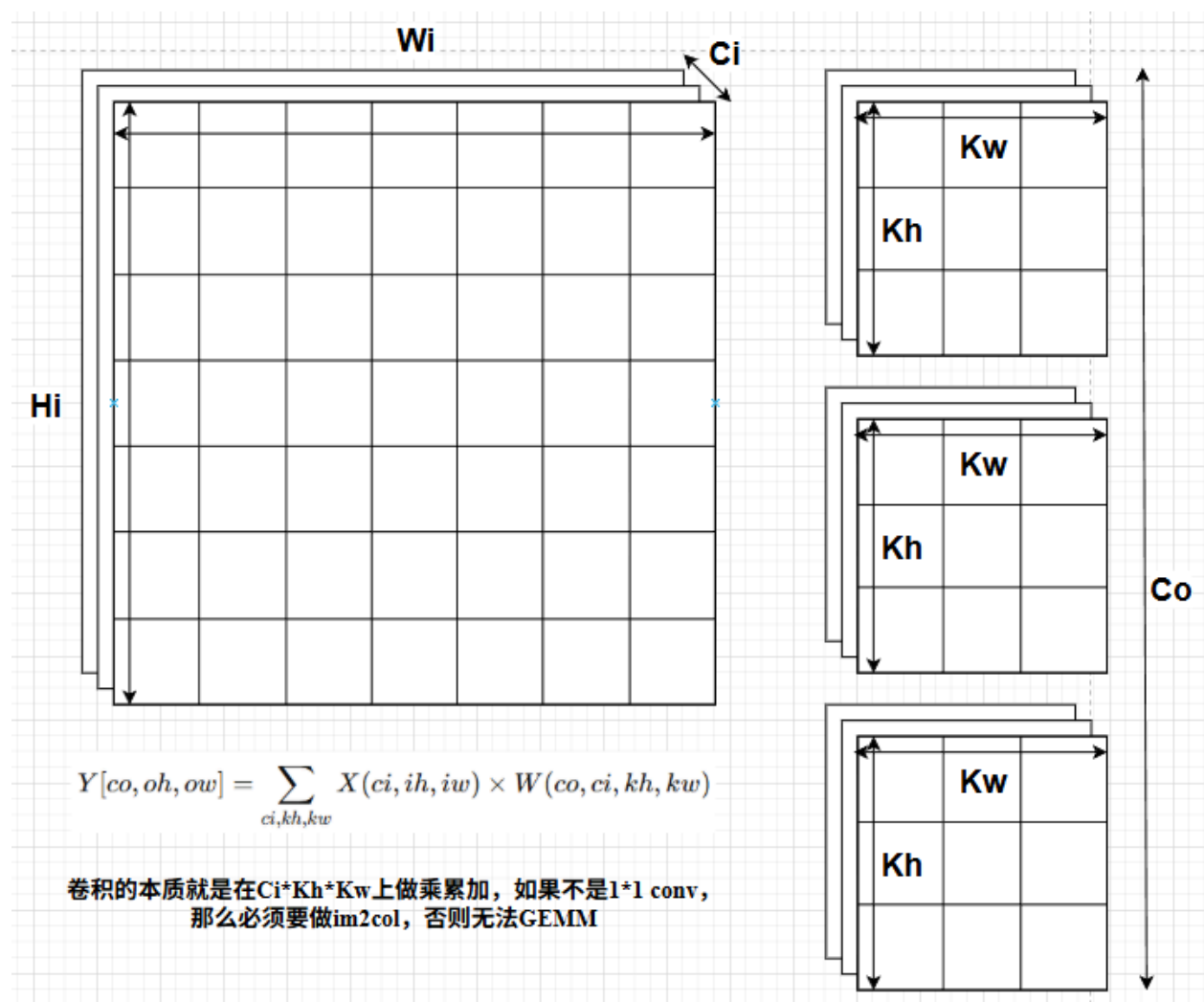
- 即 $h_i = h_o \times \text{stride} - \text{padding} + k_h \times \text{dilation}; w_i = w_o \times \text{stride} - \text{padding} + k_w \times \text{dilation}$

- **卷积补0 pad_t/b/l/r**

L2 $\rightarrow$ L1 $\rightarrow$ L0, 这里解决的**不是之前的padding, 而是L1 $\rightarrow$ L0 时的动态补0操作**

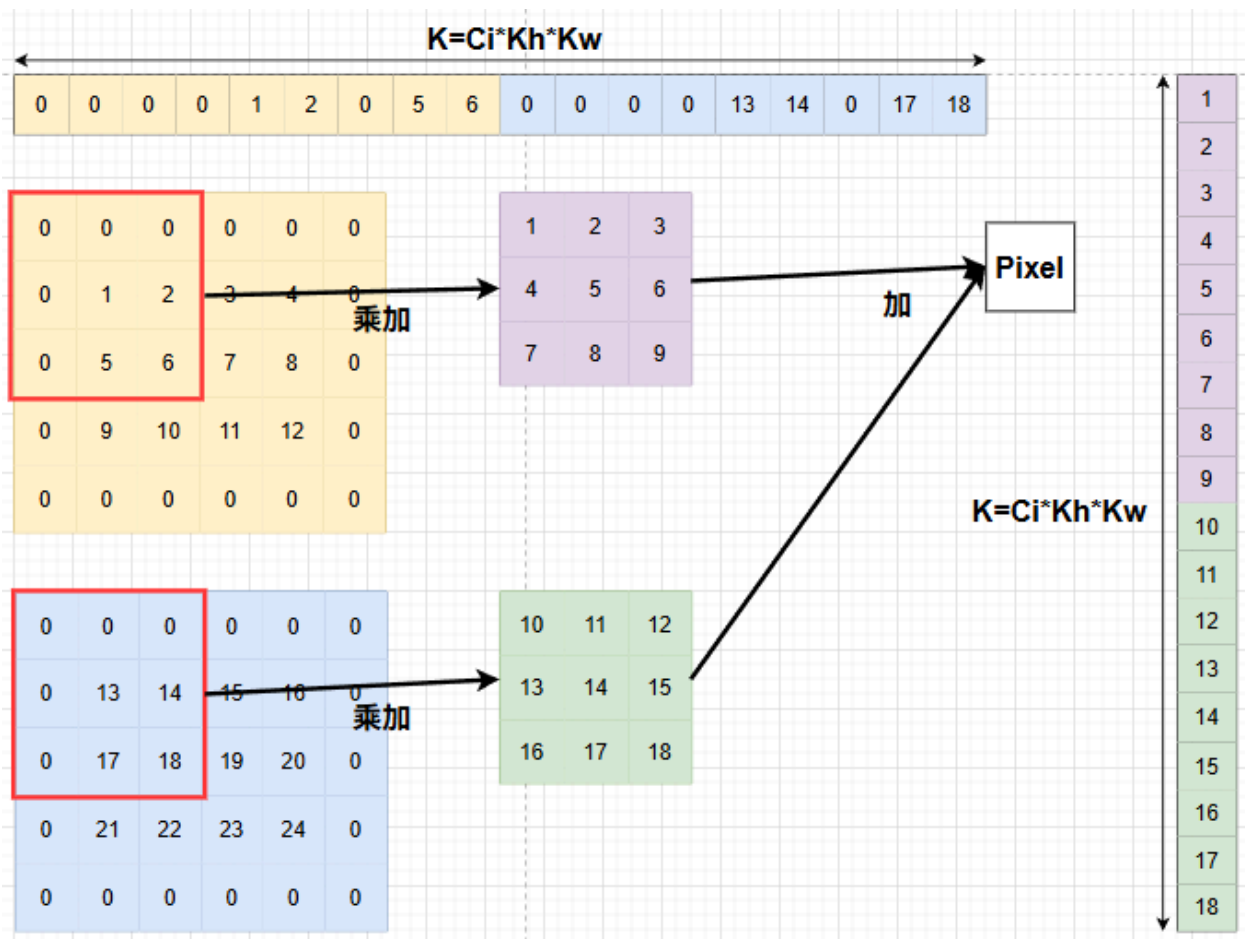
- feature map从L2 → L1中存储的都只有有效数据，为了提升空间利用率，不存在padding。
 - pad_h/w只作为参数传递，告诉硬件全局padding规则
- padding只在L1到L0中动态完成添加。他通过pad_*来完成，如果置1则需要补0。
 - 动态判断哪些地方超出了真实数据边界，需要填0，重新体现全局pad_h/w
- L2搬运到L1中可以使用全局坐标，但是L1搬运到L0中必须使用L1的局部坐标。

Convolution



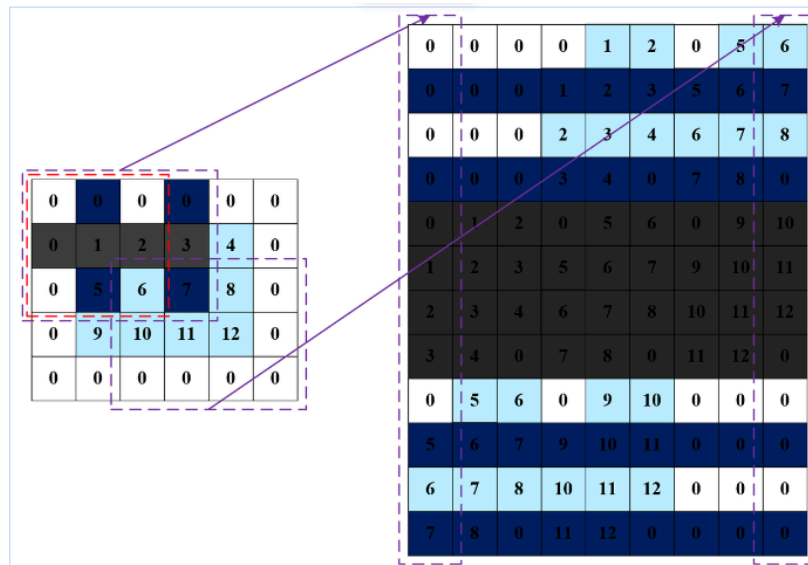
- 在不考虑stride、padding、dilation时，**输出量等于卷积结果+*bias*** ($\beta^{conv}[C_o]$) ← 不同的卷积核 (C_o) 对应的 β 量不同，所以 β 只是 C_o 的函数，换句话说：**一个输出通道共享一个偏置**。
- 对任一输出通道 $o1$ ，都有 $Y_{o1} = X * W_{o1} + \beta_{o1}$

im2col: 计算 H_o, W_o ，从输出特征图tile出发，通过 $h_o - h_i, w_o - w_i$ 的关系反推出感受野区域 (L2 → L1(tile), L1 → L0(slice))，将输入feature map取感受野区域 (receptive field region) 后，按卷积窗口展开，转换成 $M = BH_oW_o, K = C_iK_hK_w$ 的形式 (L1 → L0)

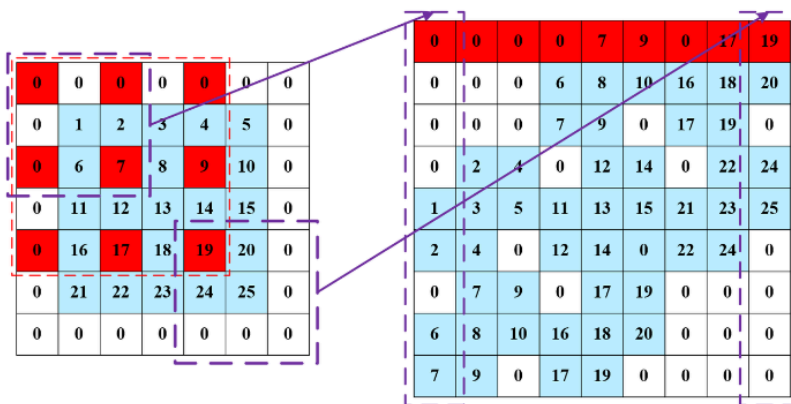


其中, $M = B \times H_o \times W_o$, 意思是output feature map中的每個元素, 而 $N = C_o$, 意思是卷積核數。 $K = C_i \times K_h \times K_w$

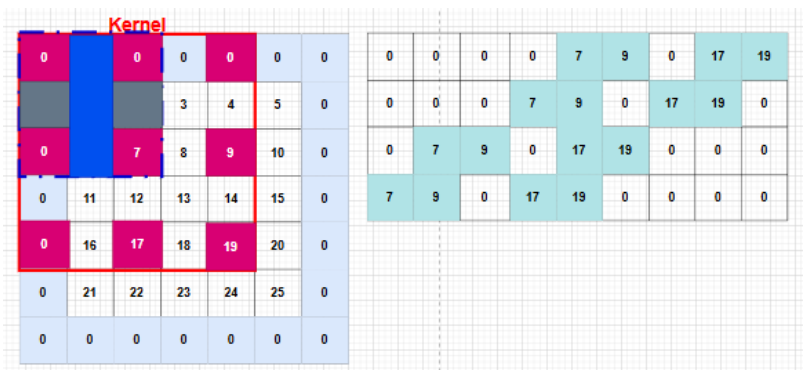
- 硬件實現: $MK \rightarrow H_o W_o C_i K_h K_w \rightarrow K_h K_w H_o W_o C_i$, 固定Kernel offset
 - 對輸入特徵圖padding
 - 左上角圈出卷積核, $Kernel$ 不動, 遍歷 $Kernel$ 的每個 K_h, K_w
 - 根據公式計算出對應的 H_o 和 W_o , 在卷積核左上角為起點畫出 $H_o \times W_o$ 的邊界框
 - 遍歷卷積核, 直到以右下角為起點
- 如果存在stride
 - 對應被採樣的 h_i, w_i 的間隔為2
 - 注意採樣時圈出的 H_o, W_o 是不包括stride計算的, 跳過的塊後續會刪除。也就是說這裡圈出的 $H_o = H_i + 2 * padding - K_{eff} + 1$, W_o 同理。



- 如果存在dilation
 - 需要圈出 $dilation * (kernel_h/wh - 1) + 1$ 大小的卷積核
 - 每次需要移動dilation

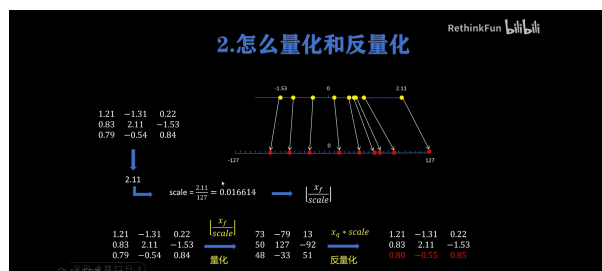


- e.g : $H = W = 5, \text{Padding} = 1, K_w = K_h = 3, \text{Dilation} = 2, \text{Stride} = 2$

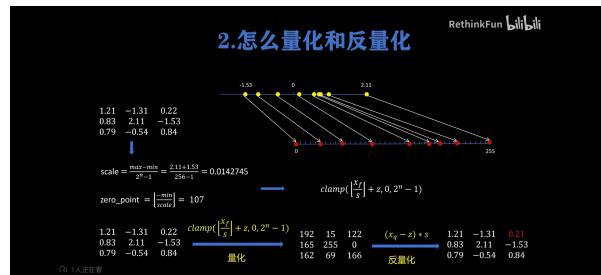


量化

- 均匀量化：固定最大值，int8映射范围为[-127,127]
 - $\text{scale} = x_{\text{fmax}} / 2^{(n-1)} - 1$
 - $\text{quant} \rightarrow [x_{\text{f}} / \text{scale}]$
 - $\text{anti-quant} \rightarrow x_{\text{q}} * \text{scale}$



- 非均匀量化：固定最小值和最大值，int8映射范围为[0,255]
 - $scale = x_{fmax} - x_{fmin} / 2^n - 1$
 - $zero = \lceil -x_{fmin} / scale \rceil$
 - $quant = \lfloor x_f / scale \rfloor + z$
 - $anti-quant = (x_q - z) * scale$



- per-tensor量化：输入特征图，因为输入特征图在Ci方向上累加，如果per-channel量化，那么输出特征图无法体现每条通道的scale，无法反量化
- per-channel量化：权重（Co方向上独立），KN右矩阵（col）
- per-token量化：MK左矩阵（row）